OCA EXAM 8

CHAPTER 5 – Class Design

By İbrahim Kürce

Brief of OCA: Oracle Certified Associate Java SE 8 Programmer I Study Guide: Exam 1Z0-808

Jeanne Boyarsky, Scott Selikoff

Class design: Working with Inheritance

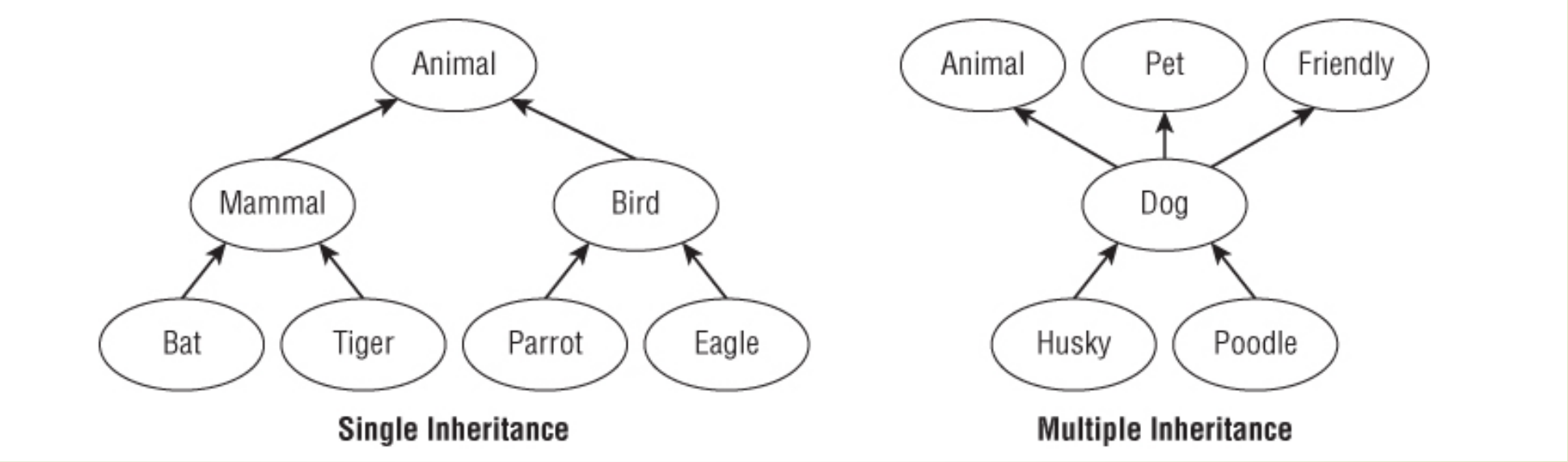
- Describe inheritance and its benefits
- Develop code that demonstrates the use of polymorphism; including overriding and object type versus reference type
- Determine when casting is necessary
- Use super and this to access objects and constructors
- Use abstract classes and interfaces

Introducing Class Inheritance

- **Inheritance** is the process by which the new child subclass automatically includes any public or protected primitives, objects, or methods defined in the parent class.
- We refer to any class that inherits from another class as a **child class**, or a **descendent** of that class.
- We refer to the class that the child inherits from as the **parent class**, or an **ancestor** of the class.
- Java supports **single inheritance**, by which a class may inherit from only one direct parent class.
- Java does allow one exception to the single inheritance rule: classes may implement **multiple interfaces**.

Class Inheritance

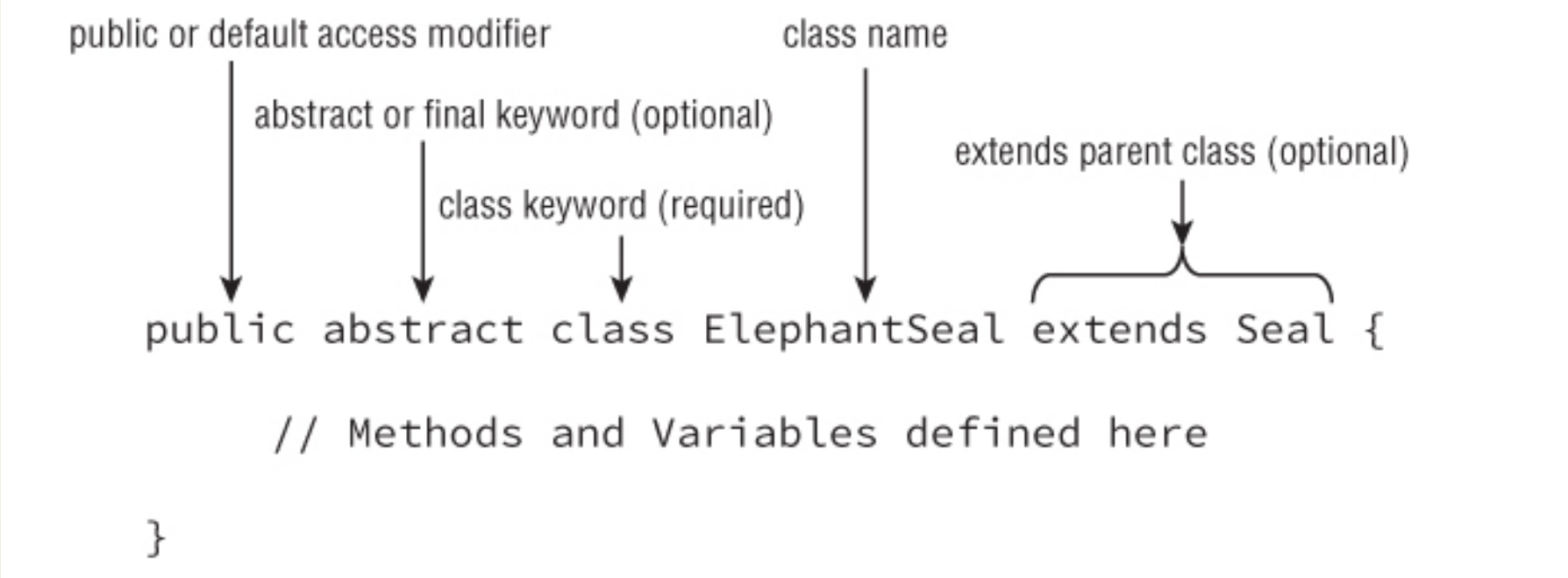
- Single inheritance vs multiple inheritance



- It is possible in Java to prevent a class from being extended by marking the class with the **final** modifier.

Extending a Class

- In Java, you can **extend** a class by adding the parent class name in the definition using the **extends** keyword.



Extending a Class

- Animal class

```
public class Animal {
  private int age;
  public int getAge() {
    return age;
  }
  public void setAge(int age) {
    this.age = age;
  }
}
```

And here are the contents of `Lion.java`:

```
public class Lion extends Animal {
  private void roar() {
    System.out.println("The "+getAge()+" year old lion says: Roar!");
  }
}
```

Applying Class Access Modifiers

- You can apply access modifiers (public, private, protected, default) to both class methods and variables.
- **Note:** For the OCA exam, you should only be familiar with **public** and **default** package-level class access modifiers. The protected and private modifiers can only be applied to inner classes, which are classes that are defined within other classes, but this is well out of scope for the OCA exam.
- **A Java file** can have many classes but at most **one public class**. In fact, it may have no public class at all.
- The rules for applying class access modifiers are identical for interfaces. There can be at most one public class or interface in a Java file.

Creating Java Objects

- In Java, all classes inherit from a single class, **java.lang.Object**.
- The key is that when Java sees you define a class that doesn't extend another class, it immediately adds the syntax **extends java.lang.Object** to the class definition.
- If you define a new class that extends an existing class, Java doesn't add this syntax.

Defining Constructors

- Every class has at least one constructor.
- In the case that no constructor is declared, the compiler will automatically insert a default no-argument constructor.
- **The first statement of every constructor is either a call to another constructor within the class, using this(), or a call to a constructor in the direct parent class, using super()**.

```
public class Animal {
  private int age;
  public Animal(int age) {
    super();
    this.age = age;
  }
}
public class Zebra extends Animal {
  public Zebra(int age) {
    super(age);
  }
  public Zebra() {
    this(4);
  }
}
```

Defining Constructors

- **The super() command may only be used as the first statement of the constructor.**

```
public class Zoo {
  public Zoo() {
    System.out.println("Zoo created");
    super(); // DOES NOT COMPILE
  }
}
public class Zoo {
  public Zoo() {
    super();
    System.out.println("Zoo created");
    super();  // DOES NOT COMPILE
  }
}
```

- If the parent class has **more than one constructor**, the child class may use any valid parent constructor in its definition.

Understanding Compiler Enhancements

- Up to now, we defined numerous classes that did not **explicitly call** the parent constructor via the super() keyword, so why did the code compile? The answer is that the **Java compiler automatically inserts a call to the no-argument constructor super()** if the first statement is not a call to the parent constructor.
- The following three class and constructor definitions are **equivalent**.

```
public class Donkey {
}
public class Donkey {
  public Donkey() {
  }
}
public class Donkey {
  public Donkey() {
    super();
    }
}
```

Constructors

- What happens if the parent class doesn't have a no-argument constructor?

```
public class Mammal {
  public Mammal(int age) {
  }
}
public class Elephant extends Mammal {  // DOES NOT COMPILE
}
```

- We must explicitly define at least one constructor, as in the following code:

```
public class Mammal {
  public Mammal(int age) {
  }
}
public class Elephant extends Mammal {
  public Elephant() {  // DOES NOT COMPILE
  }
}
```

Constructors

- This code still doesn't compile, though, because the compiler tries to insert the no-argument super() as the first statement of the constructor in the Elephant class, and there is no such constructor in the parent class.

```
public class Mammal {
  public Mammal(int age) {
  }
}
public class Elephant extends Mammal {
  public Elephant() {
    super(10);
  }
}
```

- **Note that** the class Elephant now has a no-argument constructor even though its parent class Mammal doesn't. **Subclasses may define no-argument constructors even if their parent classes do not**.

Reviewing Constructor Rules

- The first statement of every constructor is a call to another constructor within the class using **this()**, or a call to a constructor in the direct parent class using **super()**.
- **The super() call may not be used after the first statement of the constructor**.
- If **no super()** call is declared in a constructor, Java will insert a no-argument super() as the first statement of the constructor.
- If the parent doesn't have a no-argument constructor and the child doesn't define any constructors, the compiler will throw an error and try to insert a default no-argument constructor into the child class.
- If the parent doesn't have a no-argument constructor, the compiler requires an explicit call to a parent constructor in each child constructor.

Calling Constructors

- **In Java, the parent constructor is always executed before the child constructor.**

```
class Primate {
  public Primate() {
    System.out.println("Primate");
  }
}
class Ape extends Primate {
  public Ape() {
    System.out.println("Ape");
  }
}
public class Chimpanzee extends Ape {
  public static void main(String[] args) {
    new Chimpanzee();
  }
}
```

Calling Inherited Class Members

- If the parent class and child class are part of the same package, the child class may also use any **default members** defined in the parent class. Finally, a child class may never access a private member of the parent class, at least not through any direct reference.

```
class Fish {
  protected int size;
  private int age;
  public Fish(int age) {
    this.age = age;
  }
  public int getAge() {
    return age;
  }
}
public class Shark extends Fish {
  private int numberOfFins = 8;
  public Shark(int age) {
    super(age);
    this.size = 4;
  }
  public void displaySharkDetails() {
    System.out.print("Shark with age: "+getAge());
    System.out.print(" and "+size+" meters long");
    System.out.print(" with "+numberOfFins+" fins");
  }
}
```

Calling Inherited Class Members

- Consider the following alternative definition to the displaySharkDetails() method in the previous example:

```
public void displaySharkDetails() {
  System.out.print("Shark with age: "+this.getAge());
  System.out.print(" and "+this.size+" meters long");
  System.out.print(" with "+this.numberOfFins+" fins");
}
```

- In Java, you can **explicitly** reference a member of the parent class by using the super keyword.

```
public void displaySharkDetails() {
  System.out.print("Shark with age: "+super.getAge());
  System.out.print(" and "+super.size+" meters long");
  System.out.print(" with "+this.numberOfFins+" fins");
}
```

Calling Inherited Class Members

- If the child class overrides a member of the parent class, this and super could have very different effects when applied to a class member.
- **super() vs. super**
- super() and super are quite different but may be used in the same methods on the exam.
- The first, **super()**, is a statement that explicitly calls a parent constructor and may only be used in the first line of a constructor of a child class. The second, **super**, is a keyword used to reference a member defined in a parent class and may be used throughout the child class.

```
public Rabbit(int age) {
  super();
  super.setAge(10);
}
```

```
public Rabbit(int age) {
  super;  // DOES NOT COMPILE
  super().setAge(10);  // DOES NOT COMPILE
}
```

Inheriting Methods

- Inheriting a class grants us access to the public and protected members of the parent class, but also sets the stage for collisions between methods defined in both the parent class and the subclass.
- **Overriding a Method**
- What if there is a method defined in both the parent and child class?
- You may want to define a **new version** of an existing method in a child class that makes use of the definition in the parent class.
- In this case, you can **override** a method by declaring a new method **with the signature and return type** as the method in the parent class.

Inheriting Methods

- In this manner, the keywords this and super allow you to select between the current and parent version of a method.

```
public class Canine {
  public double getAverageWeight() {
    return 50;
  }
}
public class Wolf extends Canine {
  public double getAverageWeight() {

    return super.getAverageWeight()+20;
  }
  public static void main(String[] args) {
    System.out.println(new Canine().getAverageWeight());
    System.out.println(new Wolf().getAverageWeight());
  }
}
```

- What would the following code output if we removed the super keyword in the getAverageWeight() method of the Wolf class?

```
public double getAverageWeight() {
  return getAverageWeight()+20;  // INFINITE LOOP
}
```

Overriding a Method

- Overriding a method is not without limitations,
  - The method in the child class must have the same signature as the method in the parent class.
  - **The method in the child class must be at least as accessible or more accessible than the method in the parent class.**
  - **The method in the child class may not throw a checked exception that is new or broader than the class of any exception thrown in the parent class method.**
  - If the method returns a value, it must be the same or a subclass of the method in the parent class, known as **covariant return types**.

Overloading vs. Overriding

- Overloading a method and overriding a method are similar in that they both involve redefining a method using the same name.
- They differ in that an overloaded method will use a different signature than an overridden method.

```
public class Bird {
  public void fly() {
    System.out.println("Bird is flying");
  }
  public void eat(int food) {
    System.out.println("Bird is eating "+food+" units of food");
  }
}
public class Eagle extends Bird {
  public int fly(int height) {
    System.out.println("Bird is flying at "+height+" meters");
    return height;
  }
  public int eat(int food) { // DOES NOT COMPILE
    System.out.println("Bird is eating "+food+" units of food");
    return food;
  }
}
```

Overriding

- First, it violates the second rule of overriding methods: the child method must be at least as accessible as the parent. In the example, the parent method uses the **protected** modifier, but the child method uses the **private** modifier.
- The return type of the parent method and child method must be covariant. In this example, the return type of the parent method is String, whereas the return type of the child method is int.

```
public class Camel {
  protected String getNumberOfHumps() {
    return "Undefined";
  }
}
public class BactrianCamel extends Camel {
  private int getNumberOfHumps() {  // DOES NOT COMPILE
    return 2;
  }
}
```

Overriding

```
public class InsufficientDataException extends Exception {}
public class Reptile {
  protected boolean hasLegs() throws InsufficientDataException {
    throw new InsufficientDataException();
  }
  protected double getWeight() throws Exception {
    return 2;
  }
}
public class Snake extends Reptile {
  protected boolean hasLegs() {
    return false;
  }
  protected double getWeight() throws InsufficientDataException{
    return 2;
  }
}
```

Overriding

- The first method, hasLegs(), throws an exception InsufficientDataException in the parent class but doesn't throw an exception in the child class. This does not violate the third rule of overriding methods, though, as no new exception is defined.
- The second method, getWeight(), throws Exception in the parent class and InsufficientDataException in the child class. This is also permitted, as InsufficientDataException is a subclass of Exception by construction.

Overriding

```
public class InsufficientDataException extends Exception {}
public class Reptile {
  protected double getHeight() throws InsufficientDataException {
    return 2;
  }
  protected int getLength() {
    return 10;
  }
}
public class Snake extends Reptile {
  protected double getHeight() throws Exception {  // DOES NOT COMPILE
    return 2;
  }
  protected int getLength() throws InsufficientDataException { // DOES NOT COMPILE
    return 10;
  }
}
```

Overriding

- The getHeight() method in the parent class throws an InsufficientDataException, whereas the method in the child class throws an Exception. Since Exception is not a subclass of InsufficientDataException
- Next, the getLength() method doesn't throw an exception in the parent class, but it does throw an exception.

Redeclaring private Methods

- In Java, it is not possible to override a private method in a parent class since the parent method is not accessible from the child class.
- Just because a child class doesn't have access to the parent method, doesn't mean the child class can't define its own version of the method. It just means, strictly speaking, that the new method is not an overridden version of the parent class's method.
- Java permits you to redeclare a new method in the child class with the same or modified signature as the method in the parent class.

```
public class Camel {
  private String getNumberOfHumps() {
    return "Undefined";
  }
}
public class BactrianCamel extends Camel {
  private int getNumberOfHumps() {
    return 2;
  }
}
```

Hiding Static Methods

- A **hidden method** occurs when a child class defines a static method with the same name and signature as a static method defined in a parent class.
- The four previous rules for overriding a method must be followed when a method is hidden. In addition, a new rule is added for hiding a method, namely that the usage of the **static** keyword must be the same between parent and child classes.

Hiding Static Methods

- The following list summarizes the five rules for hiding a method:

1. **The method in the child class must have the same signature as the method in the parent class.**
2. **The method in the child class must be at least as accessible or more accessible than the method in the parent class.**
3. **The method in the child class may not throw a checked exception that is new or broader than the class of any exception thrown in the parent class method.**
4. **If the method returns a value, it must be the same or a subclass of the method in the parent class, known as covariant return types.**
5. **The method defined in the child class must be marked as static if it is marked as static in the parent class (method hiding). Likewise, the method must not be marked as static in the child class if it is not marked as static in the parent class (method overriding).**

Hiding Static Methods

- Some examples of the new rule:

```
public class Bear {
  public static void eat() {
    System.out.println("Bear is eating");
  }
}
public class Panda extends Bear {
  public static void eat() {
    System.out.println("Panda bear is chewing");
  }
  public static void main(String[] args) {
    Panda.eat();
  }
}
```

- The code compiles and runs without issue. The eat() method in the child class hides the eat() method in the parent class.
- Because they are both marked as static, this is not considered an overridden method.

Hiding Static Methods

- Let's contrast this with examples that **violate the fifth rule**:

```
public class Bear {
  public static void sneeze() {
    System.out.println("Bear is sneezing");
  }
  public void hibernate() {
    System.out.println("Bear is hibernating");
  }
}
public class Panda extends Bear {
  public void sneeze() {  // DOES NOT COMPILE
    System.out.println("Panda bear sneezes quietly");
  }
  public static void hibernate() {  // DOES NOT COMPILE
    System.out.println("Panda bear is going to sleep");
  }
}
```

- In this example, sneeze() is marked as static in the parent class but not in the child class. The compiler detects that you're trying to override a method that should be hidden and generates a compiler error. In the second method, hibernate() is an instance member in the parent class but a static method in the child class.

Overriding vs. Hiding Methods

- At runtime the parent version of a hidden method is always executed **if the call to the method is defined in the parent class**.

```
public class Marsupial {
  public static boolean isBiped() {
    return false;
  }
  public void getMarsupialDescription() {
    System.out.println("Marsupial walks on two legs: "+isBiped());
  }
}
public class Kangaroo extends Marsupial {
  public static boolean isBiped() {
    return true;
  }
  public void getKangarooDescription() {
    System.out.println("Kangaroo hops on two legs: "+isBiped());
  }
  public static void main(String[] args) {
    Kangaroo joey = new Kangaroo();
    joey.getMarsupialDescription();
    joey.getKangarooDescription();
  }
}
```

Overriding vs. Hiding Methods

- At runtime the child version of an overridden method is always executed for an instance regardless of **whether the method call is defined in a parent or child class method**.

```
class Marsupial {
  public boolean isBiped() {
    return false;
  }
  public void getMarsupialDescription() {
    System.out.println("Marsupial walks on two legs: "+isBiped());
  }
}
public class Kangaroo extends Marsupial {
  public boolean isBiped() {
    return true;
  }
  public void getKangarooDescription() {
    System.out.println("Kangaroo hops on two legs: "+isBiped());
  }
  public static void main(String[] args) {
    Kangaroo joey = new Kangaroo();
    joey.getMarsupialDescription();
    joey.getKangarooDescription();
  }
}
```

Overriding vs. Hiding Methods

- In this example, the isBiped() method is **overridden**, **not hidden**, in the child class. Therefore, it is replaced at runtime in the parent class with the call to the child class's method.
- Make sure you understand these examples as they show how **hidden and overridden methods are fundamentally different**. This example makes uses of polymorphism.

Creating final methods

- final methods cannot be overridden.
- This rule is in place both when you override a method and when you hide a method. In other words, you cannot hide a static method in a parent class if it is marked as final.

```
public class Bird {
  public final boolean hasFeathers() {
    return true;
  }
}
public class Penguin extends Bird {
  public final boolean hasFeathers() { // DOES NOT COMPILE
    return false;
  }
}
```

Inheriting Variables

- Java doesn't allow variables to be overridden but instead hidden.

Hiding Variables

- When you hide a variable, you define a variable with the same name as a variable in a parent class.
- **This creates two copies of the variable within an instance of the child class: one instance defined for the parent reference and another defined for the child reference**.
- As when hiding a static method, you can't override a variable; you can only hide it.
- If you're referencing the variable from within the **parent class**, the variable defined in the **parent class is used**.
- Alternatively, if you're referencing the variable from within a **child class**, the variable defined in the **child class is used**.

Hiding Variables

- You can reference the parent value of the variable with an explicit use of the **super** keyword.

```
public class Rodent {
  protected int tailLength = 4;
  public void getRodentDetails() {
    System.out.println("[parentTail="+tailLength+"]");
  }
}
public class Mouse extends Rodent {
  protected int tailLength = 8;
  public void getMouseDetails() {
    System.out.println("[tail="+tailLength +",parentTail="+super.tailLength+"]");
  }
  public static void main(String[] args) {
    Mouse mouse = new Mouse();
    mouse.getRodentDetails();
    mouse.getMouseDetails();
  }
}
```

- This code compiles without issue and outputs the following when executed: **[parentTail = 4] [tail = 8, parentTail = 4]**

Hiding Variables

- Notice that the instance of Mouse contains two copies of the tailLength variables: one defined **in the parent** and one defined **in the child**.
- The important thing to remember is that there is no notion of overriding a member variable. For example, there is no code change that could be made to cause Java to override the value of tailLength, making it the same in both parent and child.
- These rules are the same regardless of **whether the variable is an instance variable or a static variable**.

Don't Hide Variables in Practice – Real World Scenario

- Although Java allows you to hide a variable defined in a parent class with one defined in a child class, it is considered an **extremely poor coding practice**.
- Hiding variables makes the code very confusing and difficult to read, especially if you start modifying the value of the variable in both the parent and child methods, since it may not be clear which variable you're updating.

Creating Abstract Classes

- An **abstract class** is a class that is marked with the abstract keyword and **cannot be instantiated**.
- An **abstract method** is a method marked with the abstract keyword defined in an abstract class, for which no implementation is provided in the class in which it is declared.

```
public abstract class Animal {
  protected int age;
  public void eat() {
    System.out.println("Animal is eating");
  }
  public abstract String getName();
}
public class Swan extends Animal {
  public String getName() {
    return "Swan";
  }
}
```

Defining an Abstract Class

- an **abstract class** may include non abstract methods and variables, as you saw with the variable age and the method eat().
- In fact, an abstract class is not required to include any abstract methods.

```
public abstract class Cow {
}
```

```
public class Chicken {
  public abstract void peck();  // DOES NOT COMPILE
}
```

```
public abstract class Turtle {
  public abstract void swim() {}  // DOES NOT COMPILE
  public abstract int getAge() {  // DOES NOT COMPILE
    return 10;
  }
}
```

Defining an Abstract Class

- The following code snippet will not compile:

```
public final abstract class Tortoise {  // DOES NOT COMPILE
}
```

```
public abstract class Goat {
  public abstract final void chew();  // DOES NOT COMPILE
}
```

- a method may not be marked as both abstract and private.

```
public abstract class Whale {
  private abstract void sing();  // DOES NOT COMPILE
}
public class HumpbackWhale extends Whale {
  private void sing() {
    System.out.println("Humpback whale is singing");
  }
}
```

Defining an Abstract Class

- If we changed the access modified from private to protected in the parent class Whale,

```
public abstract class Whale {
  protected abstract void sing();
}
public class HumpbackWhale extends Whale {
  private void sing() {  // DOES NOT COMPILE
    System.out.println("Humpback whale is singing");
  }
}
```

- In this modified example, the code will still not compile but for a completely different reason. If you remember the rules earlier in this chapter for overriding a method, **the subclass cannot reduce the visibility of the parent method**, sing().

Creating a Concrete Class

- Abstract classes cannot be instantiated and therefore do not do much other than define static variables and methods.
- An abstract class becomes useful when it is extended by a concrete subclass. A **concrete class** is the first non abstract subclass that extends an abstract class and is required to implement all inherited abstract methods.

```
public abstract class Animal {
  public abstract String getName();

}
public class Walrus extends Animal { // DOES NOT COMPILE
}
```

Extending an Abstract Class

- Extending an abstract class with another abstract class

```
public abstract class Animal {
  public abstract String getName();

}
public class Walrus extends Animal { // DOES NOT COMPILE
}
public abstract class Eagle extends Animal {
}
```

- Abstract classes can extend other abstract classes and are not required to provide implementations for any of the abstract methods.

Extending an Abstract Class

- The following concrete class Lion must implement two methods, **getName**() and **roar**():

```
public abstract class Animal {
  public abstract String getName();
}
public abstract class BigCat extends Animal {
  public abstract void roar();
}
public class Lion extends BigCat {
  public String getName() {
    return "Lion";
  }
  public void roar() {
    System.out.println("The Lion lets out a loud ROAR!");
  }
}
```

Extending an Abstract Class

- There is one exception to the rule for abstract methods and concrete classes: a concrete subclass **is not required to provide** an implementation for an abstract method if an **intermediate** abstract class provides the implementation.

```
public abstract class Animal {
  public abstract String getName();
}
public abstract class BigCat extends Animal {
  public String getName() {
    return "BigCat";
  }
  public abstract void roar();
}
public class Lion extends BigCat {
  public void roar() {
    System.out.println("The Lion lets out a loud ROAR!");
  }
}
```

Abstract Class Definition Rules:

- Abstract classes cannot be instantiated directly.
- Abstract classes may be defined with any number, including zero, of abstract and non abstract methods.
- Abstract classes may not be marked as private or final.
- An abstract class that extends another abstract class inherits all of its abstract methods as its own abstract methods.
- The first concrete class that extends an abstract class must provide an implementation for all of the inherited abstract methods.

Abstract Method Definition Rules:

- Abstract methods may only be defined in abstract classes.
- Abstract methods **may not be declared private or final**.
- Abstract methods must not provide a method body/ implementation in the abstract class for which is it declared.
- Implementing an abstract method in a subclass follows the same rules for overriding a method. For example, the name and signature must be the same, and the visibility of the method in the subclass must be at least as accessible as the method in the parent class.

Implementing Interfaces

- Although Java doesn't allow multiple inheritance, it does allow classes to implement any number of interfaces. An **interface** is an abstract data type that defines a list of **abstract public** methods that any class implementing the interface must provide.
- **An interface can also include a list of constant variables and default methods.**

Implementing Interfaces

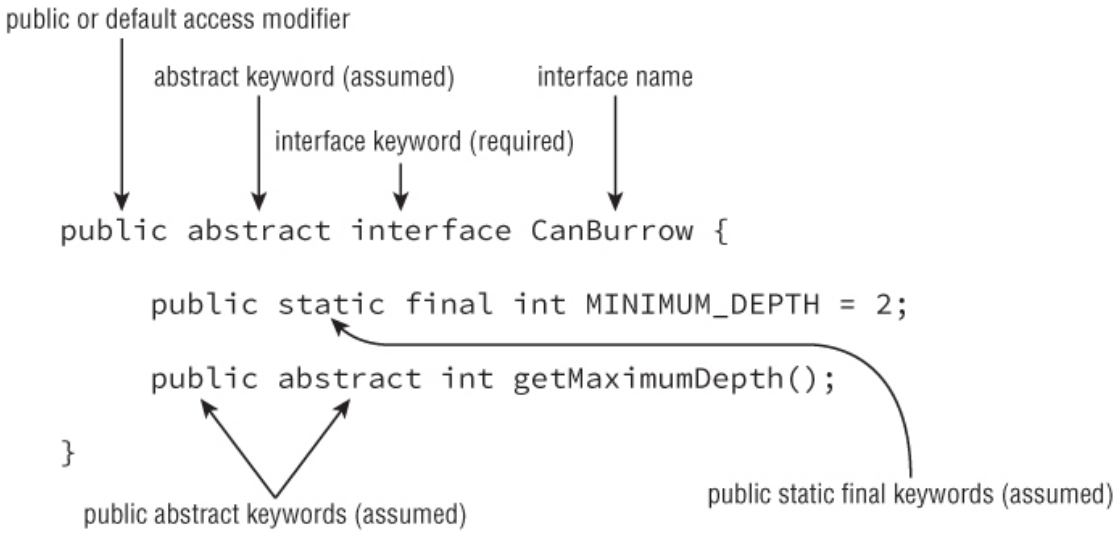


Figure 5.4 Defining an interface

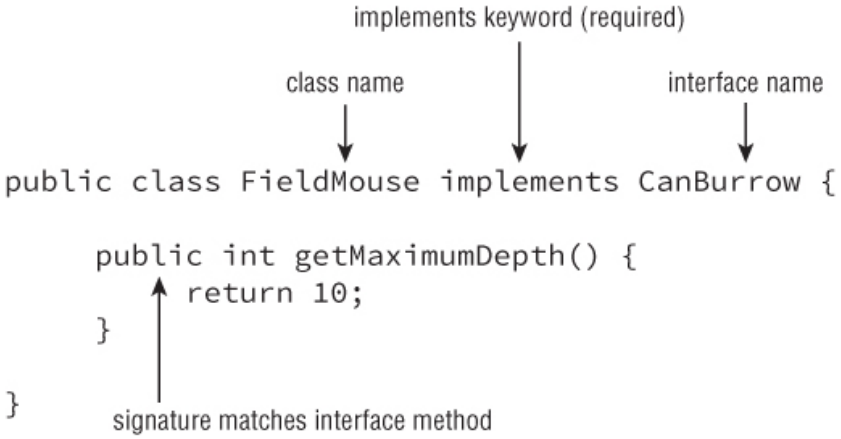


Figure 5.5 Implementing an interface

Implementing Interfaces

- abstract and public, are assumed. In other words, whether or not you provide them, the compiler will automatically insert them as part of the method definition.
- A class may implement multiple interfaces, each separated by a comma,

```
public class Elephant implements WalksOnFourLegs, HasTrunk, Herbivore {
}
```

Defining an Interface

- The following is a list of rules for creating an interface,

1. Interfaces cannot be instantiated directly.
2. An interface is not required to have any methods.
3. An interface may not be marked as final.
4. All top-level interfaces are assumed to have public or default access, and they must include the abstract modifier in their definition. **Therefore, marking an interface as private, protected, or final will trigger a compiler error**, since this is incompatible with these assumptions.
5. **All non default methods** in an interface are assumed to have the modifiers abstract and public in their definition. Therefore, marking a method as private, protected, or final will trigger compiler errors as these are incompatible with the abstract and public keywords.

Defining an Interface

- The fourth rule doesn't apply to inner interfaces, although inner classes and interfaces are **not in scope** for the OCA exam.

```
public class TestClass {
  public static void main(String[] args) {
    WalksOnTwoLegs example = new WalksOnTwoLegs();  // DOES NOT COMPILE
  }
}
public final interface WalksOnEightLegs {  // DOES NOT COMPILE
}
```

- as WalksOnTwoLegs is an interface and cannot be instantiated directly.
- Interfaces may not be marked as final

Defining an Interface

- The compiler will insert them automatically if you do not. Following two interface definitions are equivalent,

```
public interface CanFly {
  void fly(int speed);
  abstract void takeoff();
  public abstract double dive();
}
public abstract interface CanFly {
  public abstract void fly(int speed);
  public abstract void takeoff();
  public abstract double dive();
}
```

- The abstract keyword is first automatically added to the interface definition. Then, each method is prepended with abstract and public keywords.

Defining an Interface

```
private final interface CanCrawl {  // DOES NOT COMPILE
  private void dig(int depth);  // DOES NOT COMPILE
  protected abstract double depth();  // DOES NOT COMPILE
  public final void surface();  // DOES NOT COMPILE
}
```

- **The first line** doesn't compile for two reasons. First, it is marked as final, which cannot be applied to an interface since it conflicts with the assumed abstract keyword. Next, it is marked as private, which conflicts with the public or default required access for interfaces. **The second and third line** do not compile because all interface methods are assumed to be public and marking them as private or protected throws a compiler error. Finally, **the last line** doesn't compile because the method is marked as final and since interface methods are assumed to be abstract, the compiler throws an exception for using both abstract and final keywords on a method.

Inheriting an Interface

- There are two inheritance rules you should keep in mind when extending an interface:

1. An interface that **extends** another interface, as well as an abstract class that **implements** an interface, inherits all of the abstract methods as its own abstract methods.
2. The first concrete class that **implements** an interface, or **extends** an abstract class that implements an interface, must provide an implementation for all of the inherited abstract methods.

- An interface may be extended using the extend keyword. In this manner, the new child interface inherits all the abstract methods of the parent interface.

Inheriting an Interface

- Any class that implements the Seal interface must provide an implementation for all methods in the parent interfaces— in this case, getTailLength() and getNumberOfWhiskers().

```
public interface HasTail {
  public int getTailLength();
}
public interface HasWhiskers {
  public int getNumberOfWhiskers();
}
public interface Seal extends HasTail, HasWhiskers {
}
```

- The abstract class is treated in the same way as an interface extending another interface.

```
public interface HasTail {
  public int getTailLength();
}
public interface HasWhiskers {
  public int getNumberOfWhiskers();
}
public abstract class HarborSeal implements HasTail, HasWhiskers {
}
public class LeopardSeal implements HasTail, HasWhiskers {  // DOES NOT COMPILE
}
```

Classes, Interfaces, and Keywords

- Although a class can implement an interface, a class cannot extend an interface. Likewise, whereas an interface can extend another interface, an interface cannot implement another interface.

```
public interface CanRun {}
public class Cheetah extends CanRun {}  // DOES NOT COMPILE
public class Hyena {}
public interface HasFur extends Hyena {} // DOES NOT COMPILE
```

Abstract Methods and Multiple Inheritance

- What will happen if you define a class that inherits from **two interfaces** that contain the same abstract method:

```
public interface Herbivore {
  public void eatPlants();
}
public interface Omnivore {
  public void eatPlants();
  public void eatMeat();
}

public class Bear implements Herbivore, Omnivore {
  public void eatMeat() {
    System.out.println("Eating meat");
  }
  public void eatPlants() {
    System.out.println("Eating plants");
  }
}
```

- The signatures for the two interface methods eatPlants() are compatible, so you can define a class that fulfills both interfaces simultaneously:

Multiple Inheritance

- What happens if the two methods have different signatures? If the method name is the same but the input parameters are different, there is no conflict because this is considered a method overload.

```
public interface Herbivore {
  public int eatPlants(int quantity);
}
public interface Omnivore {
  public void eatPlants();
}
public class Bear implements Herbivore, Omnivore {
  public int eatPlants(int quantity) {
    System.out.println("Eating plants: "+quantity);
    return quantity;
  }
  public void eatPlants() {
    System.out.println("Eating plants");
  }
}
```

Multiple Inheritance

- If the method name and input parameters are the same but the return types are different between the two methods, the class or interface attempting to inherit both interfaces will not compile.

```
public interface Herbivore {
  public int eatPlants();
}
public interface Omnivore {
  public void eatPlants();
}
public class Bear implements Herbivore, Omnivore {
  public int eatPlants() {  // DOES NOT COMPILE
    System.out.println("Eating plants: 10");
    return 10;
  }
  public void eatPlants() {  // DOES NOT COMPILE
    System.out.println("Eating plants");
  }
}
```

Interface Variables

- Like **interface methods**, interface variables are assumed to be **public**. **Interface variables** are also assumed to be **static** and **final**.
- Two interface variables rules:

1. Interface variables are assumed to be **public, static, and final**. Therefore, marking a variable as private or protected will trigger a compiler error, as will marking any variable as abstract.
2. The value of an interface variable must be set when it is declared since it is marked as final.

```
public interface CanSwim {
  int MAXIMUM_DEPTH = 100;
  final static boolean UNDERWATER = true;
  public static final String TYPE = "Submersible";
}
public interface CanSwim {
  public static final int MAXIMUM_DEPTH = 100;
  public static final boolean UNDERWATER = true;
  public static final String TYPE = "Submersible";
}
```

Default Interface Methods

- **With the release of Java 8**, the authors of Java have introduced a new type of method to an interface, referred to as a default method.
- A **default method** is a method defined within an interface with the default keyword in which a method body is provided.
- The purpose of adding default methods to the Java language was in part to help with **code development and backward compatibility**.
- By providing a default implementation of the method, though, the interface becomes backward compatible with the **existing codebase**, while still providing those individuals who do want to use the new method with the option to override it.

Default Interface Methods

- This example defines two interface methods, one is a normal abstract method and the other a default method.

```
public interface IsWarmBlooded {
  boolean hasScales();
  public default double getTemperature() {
    return 10.0;
  }
}
```

- The following are the default interface method rules you need to be familiar with:

1. **A default method may only be declared within an interface and not within a class or abstract class.**
2. **A default method must be marked with the default keyword. If a method is marked as default, it must provide a method body.**

Default Interface Methods

3. A default method is **not assumed to be static, final, or abstract**, as it may be used or overridden by a class that implements the interface.
4. Like all methods in an interface, a default method is **assumed to be public** and will not compile if marked as private or protected.

```
public interface Carnivore {
  public default void eatMeat();  // DOES NOT COMPILE
  public int getRequiredFoodAmount() {  // DOES NOT COMPILE
    return 13;
  }
}
```

- When an interface extends another interface that contains a default method, it may choose to **ignore the default method**
- The interface may redeclare the method as abstract

Default Methods and Multiple Inheritance

- If a class implements two interfaces that have default methods with the same name and signature, the compiler will throw an error.

```
public interface Walk {
  public default int getSpeed() {
    return 5;
  }
}
public interface Run {
  public default int getSpeed() {
    return 10;
  }
}
public class Cat implements Walk, Run {  // DOES NOT COMPILE
  public static void main(String[] args) {
    System.out.println(new Cat().getSpeed());
  }
}
```

- If the subclass overrides the duplicate default methods, the code will compile without issue

```
public class Cat implements Walk, Run {
  public int getSpeed() {
    return 1;
  }
  public static void main(String[] args) {
    System.out.println(new Cat().getSpeed());
  }
}
```

Static Interface Methods

- Java 8 also now includes support for static methods within interfaces.
- These methods are defined explicitly with the **static keyword** and function nearly identically to static methods defined in classes.
- A static method defined in an interface **is not inherited** in any classes that implement the interface.

1. Like all methods in an interface, a static method is assumed to be public and will not compile if marked as private or protected.
2. To reference the static method, a reference to the name of the interface must be used.

Static Interface Methods

- The method getJumpHeight() works just like a static method as defined in a class.

```
public interface Hop {
  static int getJumpHeight() {
    return 8;
  }
}
```

```
public class Bunny implements Hop {
  public void printDetails() {
    System.out.println(getJumpHeight()); // DOES NOT COMPILE
  }
}
```

- The static interface methods are not inherited by a class

```
public class Bunny implements Hop {
  public void printDetails() {
    System.out.println(Hop.getJumpHeight());
  }
}
```

Understanding Polymorphism

- Java supports **polymorphism**, the property of an object to take on many different forms.

```
public class Primate {
  public boolean hasHair() {
    return true;
  }
}
public interface HasTail {
  public boolean isTailStriped();
}
public class Lemur extends Primate implements HasTail {
  public boolean isTailStriped() {
    return false;
  }
  public int age = 10;
  public static void main(String[] args) {
    Lemur lemur = new Lemur();
    System.out.println(lemur.age);
    HasTail hasTail = lemur;
    System.out.println(hasTail.isTailStriped());
    Primate primate = lemur;
    System.out.println(primate.hasHair());
    }

 }
```

Understanding Polymorphism

- This code compiles and executes without issue and yields the following output: **10 false true**
- The most important thing to note about this example is that only one object, Lemur, is created and referenced.
- Once the object has been assigned a new reference type, only the methods and variables available to that reference type are callable on the object **without an explicit cast.**

```
 HasTail hasTail = lemur;
 System.out.println(hasTail.age);  // DOES NOT COMPILE
Primate primate = lemur;
System.out.println(primate.isTailStriped());  // DOES NOT COMPILE
```

Object vs. Reference

- All objects are accessed by reference.
- Since all objects inherit java.lang.Object, they can all be reassigned to java.lang.Object.

```
Lemur lemur = new Lemur();
Object lemurAsObject = lemur;
```

- **Without an explicit cast** back to Lemur, as you'll see in the next section, we no longer have access to the Lemur properties of the object.
- We can summarize this principle with the following two rules:

1. The **type of the object determines** which properties exist within the object in memory.
2. The **type of the reference** to the object determines which methods and variables are accessible to the Java program.

Casting Objects

- Once we changed the reference type, we lost access to **more specific methods** defined in the subclass that still exist within the object.

```
Primate primate = lemur;
Lemur lemur2 = primate; // DOES NOT COMPILE
Lemur lemur3 = (Lemur)primate;
System.out.println(lemur3.age);
```

- Some basic rules to keep in mind when casting variables:

1. Casting an object from a subclass to a superclass doesn't require an explicit cast.
2. Casting an object from a superclass to a subclass requires an explicit cast.
3. The compiler will not allow casts to unrelated types.

Casting Objects

- Casting is not without its limitations.

```
public class Rodent {
}
public class Capybara extends Rodent {
  public static void main(String[] args) {
    Rodent rodent = new Rodent();
    Capybara capybara = (Capybara)rodent; // Throws ClassCastException at runtime
  }
}
```

- Although this code will compile without issue, it will throw a ClassCastException at runtime since the object being referenced is not an instance of the Capybara class.
- Note:Topic is out of scope for the OCA exam, keep in mind that the **instanceof** operator can be used to check whether an object belongs to a particular class

```
if(rodent instanceof Capybara) {
  Capybara capybara = (Capybara)rodent;
}
```

Virtual Methods

- The most important feature of polymorphism is to support virtual methods. A **virtual method** is a method in which the specific implementation is not determined until runtime.
- In fact, all **non-final, non-static, and non-private** Java methods are considered virtual methods, since any of them **can be overridden** at runtime.
- What makes a virtual method special in Java is that if you call a method on an object that overrides a method, you get the overridden method, even if the call to the method is on a parent reference or within the parent class.

Virtual Methods

- We'll illustrate this principle with the following example:

```
public class Bird {
  public String getName() {
    return "Unknown";
  }
  public void displayInformation() {
    System.out.println("The bird name is: "+getName());
  }
}
public class Peacock extends Bird {
  public String getName() {
    return "Peacock";
  }
  public static void main(String[] args) {
    Bird bird = new Peacock();
    bird.displayInformation();
  }
}
```

- The bird name is: Peacock

Polymorphic Parameters

- One of the most useful applications of polymorphism is the ability to pass instances of a subclass or interface to a method. For example, you can define a method that takes an **instance of an interface** as a parameter.
- This property is referred to as **polymorphic parameters** of a method.

Polymorphic Parameters

```
public class Reptile {
  public String getName() {
    return "Reptile";
  }
}
public class Alligator extends Reptile {
  public String getName() {
    return "Alligator";
  }
}

public class Crocodile extends Reptile {
  public String getName() {
    return "Crocodile";
  }
}
public class ZooWorker {
  public static void feed(Reptile reptile) {
    System.out.println("Feeding reptile "+reptile.getName());
  }
  public static void main(String[] args) {
    feed(new Alligator());
    feed(new Crocodile());
    feed(new Reptile());
  }
}
```

This code compiles and executes without issue, yielding the following output:

```
Feeding: Alligator
Feeding: Crocodile
Feeding: Reptile
```

Polymorphism and Method Overriding

- Let's conclude this chapter by returning to the last three rules for method overriding to demonstrate how polymorphism requires them to be included as part of the Java specification.
- The first rule is that an overridden method must be at least as accessible as the method it is overriding.

```
public class Animal {
  public String getName() {
    return "Animal";
  }
}
public class Gorilla extends Animal {
  protected String getName() {  // DOES NOT COMPILE
    return "Gorilla";
  }
}
public class ZooKeeper {
  public static void main(String[] args) {
    Animal animal = new Gorilla();
    System.out.println(animal.getName());
  }
}
```

Polymorphism and Method Overriding

- The Java compiler disallows overriding methods with new or broader exceptions.
- Finally, overridden methods must use covariant return types for the same kinds of reasons as just discussed.